P1205R0
Teleportation via `co_await`

Published Proposal, 2018-09-28

This version:
http://wg21.link/P1205R0

Authors:
Olivier Giroux (NVIDIA)
JF Bastien (Apple)

Audience:
SG1, CWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/jfbastien/papers/blob/master/source/P1205R0.bs

Table of Contents

§ 1. Issues

The C++ Coroutine TS [N4736] has issues 31 and 32 listed in [P0664R5]:

> **31.** Add a note warning about thread switching near await and/or `coroutine_handle` wording.
>
> Add a note warning about thread switching near await and/or `coroutine_handle` wording
>
> **32.** Add a normative text making it UB to migrate coroutines between certain kind of execution agents.
>
> Add a normative text making it UB to migrate coroutines between certain kind of execution agents. Clarify that migrating between `std::thread`s is OK. But migrating between CPU and GPU is UB.

§ 2. Discussion

Using `co_await`, one can teleport a suspended execution between execution agents:

```
thread::id get_an_id() {

  // here: acquire a lock, read thread_local

  co_yield std::this_thread::get_id(); //< one result

  // UB: release the lock, reuse the same thread_local

  co_return std::this_thread::get_id(); //< different result
}
```

We say "teleport" here because the code that relocates the coroutine is outside the coroutine, in a possibly unrelated part of the program. This teleportation can take your coroutine to many interesting places, for example:

1. the thread that runs `main`
2. threads from `std::thread` / `std::async`
3. elemental functions of `std::par`, `std::par_unseq`, `std::unseq` algorithms
4. global / `thread_local` constructors (see note)
5. global / `thread_local` / `static` destructors (see note)
6. functions registered with `at_exit` / `quick_exit`
7. signal handlers

8. future `fibers_context` of [P0876R3]

Note that it is presently implementation-defined whether many of these functions run in a specific thread, a single thread, or in many unspecified threads—see [CWG2046].

§ 3. Proposed Resolution

After [N4736] [**dcl.fct.def.coroutine**] ¶6:

> A suspended coroutine can be resumed to continue execution by invoking a resumption member function of an object of type `coroutine_handle<P>` associated with this instance of the coroutine. The function that invoked a resumption member function is called *resumer*. Invoking a resumption member function for a coroutine that is not suspended results in undefined behavior.

Add ¶7:

> Resuming a coroutine on an execution agent other than the one it was suspended on has implementation-defined behavior unless both are instances of `std::thread`. [*Note*: a coroutine that is moved this way should avoid the use of `thread_local` or `mutex` objects. — *End note*.]

§ References

§ Informative References

[CWG2046]
Dinka Ranns. Incomplete thread specifications. 17 November 2014. C++17. URL: https://wg21.link/cwg2046

[N4736]
Gor Nishanov. Working Draft, C ++ Extensions for Coroutines. 20180331. URL: https://wg21.link/n4736

[P0664R5]
Gor Nishanov. C++ Coroutine TS Issues. 24 June 2018. URL: https://wg21.link/p0664r5

[P0876R3]

Oliver Kowalke, Nat Goodspeed. fiber_handle - fibers without scheduler. 8 June 2018. URL: https://wg21.link/p0876r3